Projekt: https://github.com/ronaldsieber/SmartMeterMonitor
Lizenz: MIT
Autor: Ronald Sieber

SmartMeterMonitor

Dieses Projekt implementiert eine Applikation zum Auslesen von Stromzählern (SmartMeter) über die **Modbus/RTU**-Schnittstelle. Zusätzlich kann optional ein Gaszähler-Sensor über die **S0-Schnittstelle** (GPIO) eingebunden werden. Die erfassten Messwerte werden als JSON-Datensätze über **MQTT** an einen Broker publiziert und sind so in Smart-Home-Systeme oder für Datenbanken (z.B. InfluxDB) nutzbar. Die Applikation ist primär für den Einsatz auf einem ***sysWORXX Pi-AM62x*** mit ***Smart Metering HAT*** unter Linux konzipiert und für einen zuverlässigen 24/7-Dauerbetrieb ausgelegt. Alternativ kann die Software auch auf einem **Raspberry Pi** mit **RS485 HAT** oder **USB RS485** Adapter genutzt werden. Support für die S0-Schnittstelle bietet jedoch nur der *Smart Metering HAT*:

Controller Board	Meter Adapter	RS485 SmartMeter	S0 ImpulseSensor
sysWORXX Pi-AM62x	Smart Metering HAT	Ja	Ja
sysWORXX Pi-AM62x	USB RS485 Adapter	Ja	Nein
Raspberry Pi	RS485 HAT	Ja	Nein
Raspberry Pi	USB RS485 Adapter	Ja	Nein

[sysWORXX Pi-AM62x + Smart Metering HAT]

Projektübersicht

Unterstützte SmartMeter (Modbus/RTU)

Gerät	Beschreibung
Eastron SDM230	1-phasiger Stromzähler
Eastron SDM630	3-phasiger Stromzähler

Die SmartMeter werden über einen RS485 Adapter (Modbus/RTU) mit dem Controller Board (*sysWORXX Pi-AM62x* oder *Raspberry Pi*) verbunden. Der Adapter nutzt eine serielle Schnittstelle (z.B. */dev/ttyS3*, */dev/ttyUSB0*).

[SmartMeter – SDM230]

Unterstützte Gaszähler-Sensoren (S0/GPIO)

Gerät	Beschreibung
ES-GAS-2	Gaszähler-Sensor mit S0-Impulsausgang

Der S0-Impulsausgang des Gaszähler-Sensors ist mit einem GPIO-Pin des Controller Boards verbunden (z.B. *GPIO 0*). Das *Smart Metering HAT* für den *sysWORXX Pi-AM62x* besitzt hierfür einen entsprechenden S0-Eingang.

[GasMeter Sensor – ES-GAS-2]

Funktionsweise

1. **Konfiguration laden:** Beim Start wird die Konfigurationsdatei (typ. *config.ini*) eingelesen
2. **MQTT-Verbindung:** Verbindungsaufbau zum konfigurierten MQTT-Broker
3. **Zyklische Abfrage:** In konfigurierbaren Intervallen werden alle SmartMeter über Modbus ausgelesen sowie ggf. der Zählerstand des Gaszählers
4. **Datenübertragung:** Die Messwerte werden als JSON-Record an den MQTT-Broker publiziert
5. **KeepAlive:** Regelmäßiges Senden von KeepAlive-Nachrichten zur Überwachung der Applikation

Messwerte

SDM230 (1-phasig)

Messgröße	**Einheit**	**JSON Key**
Spannung L1	V	"Voltage_L1":
Strom L1	A	"Current_L1":
Leistung L1	W	"Power_L1":
Frequenz	Hz	"Frequency":
Zählerstand	kWh	"Energy":

SDM630 (3-phasig)

Messgröße	**Einheit**	**JSON Key**
Spannung L1, L2, L3	V	"Voltage_L1":, "Voltage_L2":, "Voltage_L3":
Strom L1, L2, L3	A	"Current_L1":, "Current_L2":, "Current_L3":
Leistung L1, L2, L3	W	"Power_L1":, "Power_L2":, "Power_L3":
Summen-Strom	A	"Current_Sum":
Summen-Leistung	W	"Power_Sum":
Frequenz	Hz	"Frequency":
Zählerstand	kWh	"Energy":

Gaszähler-Sensor

Messgröße	Einheit	JSON Key
Impulszähler	Impulse	"Counter":
Zählerstand	m³	"MeterValue":
Leistung	W	"Power":
Energie	kWh	"Energy":

Aufbau des Systems

```
+--------------------+
|                    |
|     MQTT Broker    |
|                    |
+--------------------+
          |
          | LAN/WLAN
          |
+--------------------+    +--------------------+               +------------+
|                    |    |                    |  Modbus/RTU   |   SDM230   |
|                    |    |                    |---------------|   SDM630   |
|      sysWORXX      |    |      sysWORXX      |    RS485      +------------+
|      Pi-AM62x      |----| Smart Metering HAT |
|                    |    |                    |  DI/Impulse   +------------+
|                    |    |                    |---------------|  ES-GAS-2  |
+--------------------+    +--------------------+      S0       +------------+
```

Konfiguration

Die Konfiguration erfolgt über eine INI-Datei (typisch *`config.ini`*). Die Datei ist in mehrere Sektionen unterteilt.

Sektion [Settings]

Diese Sektion beinhaltet globale Einstellungen, die für die gesamte Applikation gelten.

Parameter	Pflicht	Beschreibung
MbInterface	Ja	Modbus-Schnittstelle (z.B. `/dev/ttyS0`, `/dev/ttyUSB0`) (sysWORXX Pi-AM62x mit Smart Metering HAT: `/dev/ttyS3`)
Baudrate	Ja	Baudrate der seriellen Schnittstelle (z.B. `9600`)
QueryInterval	Ja	Abfrageintervall in Sekunden (z.B. `60`)
InterDevPause	Nein	Pause zwischen Abfrage der Geräte in Sekunden (z.B. `0.25`)
MqttBroker	Ja	IP-Adresse und Port des MQTT-Brokers (z.B. `192.168.3.200:1883`)
ClientName	Nein	MQTT Client-Name (Standard: `SmartMeterMon_<uid>`)
UserName	Nein	MQTT Benutzername für Authentifizierung
Password	Nein	MQTT Passwort für Authentifizierung
StatTopic	Ja	MQTT Topic für Status-Nachrichten (Bootup, KeepAlive)
DataTopic	Ja	MQTT Topic-Template für Messdaten darf Platzhalter `<label>` und `<id>` enthalten (siehe unten)
JsonInfoLevel	Nein	Detailgrad der JSON-Ausgabe (siehe unten)

Platzhalter:
- `<label>` = wird durch das Label des Geräts ersetzt
- `<id>` = wird durch die ModbusID (3-stellig) ersetzt
- `<uid>` = wird durch eine von der Applikation zur Laufzeit erzeugte UUID ersetzt

JsonInfoLevel:
- 0 = Messwerte + Gerätetyp
- 1 = Messwerte + Gerätetyp + Zeitstempel
- 2 = Messwerte + Gerätetyp + Zeitstempel + Detail-Informationen (Label, ModbusID, Range)

Sektion [DeviceNNN]

Für jedes SmartMeter wird eine eigene Sektion angelegt. Die Sektionen müssen fortlaufend nummeriert sein (`Device001`, `Device002`, …).

Parameter	Pflicht	Beschreibung
Type	Ja	Gerätetyp (`SDM230` oder `SDM630`)
ModbusID	Ja	Modbus Slave-ID (1-247)
Label	Nein	Bezeichnung der Messstelle (für Topic und JSON)

Sektion [GasMeter]

Optionale Konfiguration für einen Gaszähler mit S0-Schnittstelle.

Parameter	Pflicht	Beschreibung
SensorGpio	Ja	GPIO-Pin für den Gaszähler/S0-Sensor (0-63) (sysWORXX Pi-AM62x mit Smart Metering HAT: `0`)
Elasticity	Nein	"Elastizitäsfaktor" zur Berechnung des Wertes *"Energy"* (1-100, Standard: `1`, Beschreibung siehe unten)
CounterDelta	Nein	Mindest-Impulsdifferenz für Aktualisierung (1-10, Standard: `1`, Beschreibung siehe unten)
VolumeFactor	Ja	Umrechnungsfaktor Impulse → Volumen [m³] (z.B. `0.01` für Gaszähler vom Typ "Elster", 1 Imp = 0.01 m³)
EnergyFactor	Ja	Umrechnungsfaktor Volumen → Energie [kWh/m³] (z.B. `0.1062255`, siehe unten)
RetainFile	Nein	Datei zum Speichern des Zählerstands (Persistenz) (z.B. `/home/smm/SmartMeterMonitor/GasMeterRetain.dat`, siehe unten)
Label	Nein	Bezeichnung der Messstelle (für Topic und JSON)

RetainFile:
Das Retain-File dient sowohl zur Anfangsinitialisierung des Zählerwertes als auch für dessen zyklische persistente Speicherung. Beim Start der Applikation wird der im Retain-File eingetragene Wert als Anfangszählerwert übernommen. Unmittelbar vor der Inbetriebnahme kann dort der aktuelle Zählerstand des mechanischen Zählwerkes eingetragen werden. Dieser wird dann beim Start der Applikation gelesen und als Anfangszählerwert übernommen. Damit wird erreicht, dass die Applikation denselben Wert verwendet wie das mechanische Zählwerk. Das Retain-File besitzt folgenden Aufbau:

```
MeterValue = 1234.56
```

EnergyFactor:

Der *'EnergyFactor'* gibt die Energiemenge an, die einem Zählerimpuls entspricht. Diese berechnet sich wie folgt:

*1 Imp = Brennwert * Zustandszahl * VolumeFactor*

Dabei ist:
- Brennwert: Energiegehalt je m³ Gas (vom Energieversorger erfragen oder auf Rechnung ablesen)
  typisch: ca. 10..12 kWh/m³
- Zustandszahl: Faktor, der die örtlichen Temperatur- und Druckverhältnisse berücksichtigt (vom Energieversorger erfragen oder auf Rechnung ablesen)
  typisch: 0,8..1,0
- VolumeFactor: Umrechnungsfaktor Impulse in Volumen [m³] (auf Typenschild des Gaszählers ablesen)
  z.B.: 0.01 für Gaszähler vom Typ "Elster", 1 Imp = 0.01 m³

Beispiel:
1 Imp = 0.01 m3 => 11.5 kWh/m3 * 0.9237 * 0.01 = 0.1062255 kWh/Imp

Elasticity und **CounterDelta:**

Die beiden Parameter *'Elasticity'* und *'CounterDelta'* steuern die Berechnung der aktuellen Wärmeleistung (Parameter *'Power:' in [W]* im JSON Record).

Standardwerte:

```
Elasticity = 1
CounterDelta = 1
```

Diese Standardwerte sind für große Verbraucher wie Gasheizungen in Ein- oder Mehrfamilienhäusern ausreichend, da hier ohnehin mehrere Zählerimpulse je Messintervall auftreten.

Monitoring von kleinen Verbrauchern:

Bei kleineren Geräten wie z.B. einem Gasherd ist der Verbrauch oft so gering, dass trotz kontinuierlicher Abnahme über mehrere Messintervalle hinweg kein Zählimpuls auftritt. Der Gaszähler-Sensor ist ein Reed-Kontakt, der pro Umdrehung des Zählwerks einen Impuls liefert. Bei geringem Verbrauch (z.B. Gasherd) führt dies zu folgenden Effekten:

- Über längere Zeit wird kein Impuls registriert (Power = 0 oder -1)
- Bei einem Impuls wird die gesamte Energie einem kurzen Zeitfenster (dem aktuellen Messintervall) zugeordnet, was einen unrealistisch hohen Leistungsspitzenwert ergibt

Beispiel bei 60 Sekunden Abfrageintervall:

(3600 sec/h ÷ 60 sec) * 0.1 kWh/Imp = 6 kW Spitzenwert

Parameter 'Elasticity'

Definiert die maximale Anzahl von Abfrageintervallen, nach deren Ablauf ein gültiger Leistungswert berechnet und übertragen wird.

Verhalten:

- Solange weder ein Zählerimpuls auftritt noch die als *Elasticity* definierte Anzahl von Intervallen erreicht ist, wird *'Power:'* auf -1 gesetzt.
  Dies signalisiert: "Kein gültiger Wert vorliegend"

- Bei Erreichen der als *Elasticity* definierten Anzahl von Intervallen OHNE Zählerimpuls wird *'Power:'* auf 0 gesetzt.
  Dies signalisiert: "Tatsächlich kein Verbrauch"

- Bei einem Zählerimpuls wird die Leistung über die gesamte verstrichene Zeitspanne seit der letzten Berechnung gemittelt.

Wichtig: Die Berechnung basiert auf der tatsächlich verstrichenen Zeit, nicht auf einer festen Anzahl von Intervallen. Je später ein Impuls innerhalb des *Elasticity*-Fensters auftritt, desto länger die Zeitbasis und desto niedriger der berechnete Durchschnittswert.

Parameter 'CounterDelta'

Definiert, um wie viele Zählerschritte sich der Wert erhöhen muss, bevor ein gültiger Leistungswert übertragen wird. Dieser Parameter ermöglicht eine Glättung über mehrere Impulse hinweg, unabhängig von der *Elasticity*-Konfiguration.

Bedingungen für Neuberechnung

Ein gültiger Leistungswert (*'Power:'* >= 0) wird berechnet, wenn MINDESTENS EINE der folgenden Bedingungen erfüllt ist:

(1) Der Zählerwert hat sich um mindestens *'CounterDelta'* erhöht
- ODER -
(2) Die *'Elasticity'*-Anzahl von Abfrageintervallen wurde erreicht

Berechnungsformel

Die Leistung wird wie folgt berechnet:

```
Power [W] = (CounterDiff * EnergyFactor) * (3600 / TimeSpan_sec) * 1000
```

Dabei ist:
- CounterDiff: Anzahl Impulse seit letzter Berechnung
- EnergyFactor: Umrechnungsfaktor Impulse zu kWh (typisch etwa 0,1 kWh/Imp)
- TimeSpan_sec: Sekunden seit letzter Berechnung

Empfohlene Einstellungen

Für Kleinverbraucher (z.B. Gasherd):

```
Elasticity = 3
CounterDelta = 2
```

Für Großverbraucher (z.B. Gasheizung):
Standardwerte sind ausreichend (*Elasticity = 1*, *CounterDelta = 1*)

Hinweis: Die Einstellungen sollten sich am kleinsten Verbraucher der Anlage orientieren. Für große Verbraucher sind die Parameter weniger relevant, da diese ohnehin häufig Impulse erzeugen.

Praktische Richtwerte

Eine typische Gasheizung eines Zweifamilienhauses erzeugt bei aktivem Brenner ca. 0,03 m³/min Verbrauch (ca. 20 kW), was etwa 3 Impulsen pro 60-Sekunden-Abfrageintervall entspricht.

Beispiel-Konfiguration

```
[Settings]
MbInterface = /dev/ttyS3                ; RS485 Modbus/RTU for SmartMeter
Baudrate = 9600
QueryInterval = 60                      ; [sec]
InterDevPause = 0.25                    ; [sec]
MqttBroker = 192.168.3.200:1883
StatTopic = SmartMeter/Status           ; app globally, not for a device
DataTopic = SmartMeter/Data/<label>     ; is individualized for each device
JsonInfoLevel = 2

[Device001]
Type = SDM630
ModbusID = 10
Label = Flat_1

[Device002]
Type = SDM630
ModbusID = 20
Label = Flat_2

[Device003]
Type = SDM230
ModbusID = 21
Label = HeatPump

[GasMeter]
SensorGpio = 0                          ; GPIO Pin for GasMeter/S0-Sensor
Elasticity = 3
CounterDelta = 2
VolumeFactor = 0.01                     ; 1 Imp = 0.01 m3
EnergyFactor = 0.1062255                ; value got from energy supplier
RetainFile = /home/smm/GasMeter.dat
Label = NaturalGasMeter
```

Kommandozeile für Programmaufruf

Der *SmartMeterMonitor* ist eine Kommandozeilen-Tool, das typischerweise als Dienst beim Systemstart gestartet wird. Das Tool unterstützt folgende Kommandozeilenparameter:

```
SmartMeterMonitor [OPTION]

OPTION:
    -c=<cfg_file>   Path/Name of Configuration File
    -l=<msg_file>   Logs all Messages sent via MQTT to specified file
                    (the file is opened always in APPEND mode)
    -o              Run in Offline Mode, without MQTT connection
    -v              Run in Verbose Mode (extended logging output)
    --help          Shows Help Screen
```

Beispielaufruf

```
./SmartMeterMonitor -c=config.ini -v
```

Start-Skript

Das Beispiel-Skript `run_SmartMeterMonitor.sh` realisiert folgende Aufgaben:

- Setzen des RS485-Modus für die serielle Schnittstelle `/dev/ttyS3`
  (nur notwendig bei *sysWORXX Pi-AM62x* mit *Smart Metering HAT*)
- Setzen der Bibliothekspfad-Variable `LD_LIBRARY_PATH` für `libmodbus.so`
  (siehe Abschnitt *"Bauen der Applikation"* weiter unten)
- Starten der Applikation

```
#!/bin/bash

rs485-setup /dev/ttyS3

export LD_LIBRARY_PATH=/home/smm/bin/lib
./SmartMeterMonitor/SmartMeterMonitor -c=config.ini
```

Device Tree für sysWORXX Pi-AM62x mit Smart Metering HAT

Der *sysWORXX Pi-AM62x* unterstützt verschiedene IO-Belegung für den 40poligen Pin-Header. Die Steuerung des Pin-Muxings erfolgt Linux-typisch mit Hilfe des Device Trees, zusammen mit Device Tree Overlays (DTBO), die separat geladen werden. Um die für den ***Smart Metering HAT*** notwendige IO-Belegung bereitzustellen, ist das entsprechende *Smart Metering HAT* Overlay zu laden. Dazu sind als Benutzer root folgende Befehle auszuführen:

```
sudo dtbo-setup set k3-am625-systec-sysworxx-pi-hat-smart-metering.dtbo
reboot
```

Mit aktivem *Smart Metering HAT* Overlay gelten folgende Einstellungen für die Konfigurationsdatei (typ. `config.ini`):

```
[Settings]
MbInterface = /dev/ttyS3     ; RS485 Modbus/RTU Interface for SmartMeter

[GasMeter]
SensorGpio = 0               ; GPIO Pin for GasMeter/S0-Sensor
```

MQTT-Nachrichten

Bootup-Nachricht

Bei jedem Start wird eine Bootup-Nachricht an das konfigurierte `StatTopic` publiziert (z.B. Topic: `SmartMeter/Status/Bootup`):

```
{
  "Version": "1.00",
  "TimeStamp": 1704067200,
  "TimeStampFmt": "2025/01/01 00:00:00",
  "ClientName": "SmartMeterMon_A1B2C3D4",
  "KeepAlive": 30
}
```

Messdaten-Nachricht

Die Messdaten werden an das konfigurierte `DataTopic` publiziert (z.B. Topic: `SmartMeter/Data/HeatPump`):

```
{
  "TimeStamp": 1704067260,
  "TimeStampFmt": "2025/01/01 00:01:00",
  "Type": "SDM230",
  "ModbusID": 21,
  "Label": "HeatPump",
  "Voltage_L1": 230.5,
  "Current_L1": 2.35,
  "Power_L1": 541.2,
  "Frequency": 50.01,
  "Energy": 12345.67
}
```

Bauen der Applikation

Projektstruktur

Die Projektstruktur für die Applikation *SmartMeterMonitor* untergliedert sich in folgende Hauptkomponenten:

- *SmartMeterMonitor/* Applikationsspezifische Sourcen
- *libmodbus/* Bibliothek *'libmodbus'* für Modbus (Nutzung als dynamische Bibliothek)
- *Mqtt/* Bibliothek *'paho-mqtt-embedded-c'* für MQTT
- *configparser/* Bibliothek *'configparser'* als INI-File Parser

```
SmartMeterMonitor/
  |
  +-- libmodbus/                            # <--- Separate Donload!
  |      |
  |      +-- autogen.sh                     # Build Step (1)
  |      +-- configure                      # Build Step (2)
  |      +-- Makefile                       # Build Step (3)
  |      |
  |      +-- ...
  |
  +-- Mqtt/
  |      |
  |      +-- Mqtt_Transport/
  |      |      +-- MqttTransport.h
  |      |      +-- MqttTransport_Posix.c
  |      |
  |      +-- paho_mqtt_embedded_c/
  |            +-- ...
  |
  +-- configparser/
  |      |
  |      +-- ...
  |
  +-- SmartMeterMonitor/
  |      |
  |      +-- Makefile                       # Build Step (4)
  |      |
  |      +-- SmartMeter/
  |      |      +-- SmartMeter.h
  |      |      +-- SDM230.cpp/h
  |      |      +-- SDM630.cpp/h
  |      |
  |      +-- GasMeter/
  |      |      +-- GasMeter.cpp/h
  |      |      +-- GMSensIf.h
  |      |      +-- GMSensIf_sysPi.cpp
  |      |
  |      +-- Main.cpp
  |      +-- CfgReader.cpp/h
  |      +-- LibMqtt.cpp/h
  |      +-- LibSys.cpp/h
  |      +-- Trace.cpp/h
  |
  +-- config.ini                            # Example Configuration
  +-- run_SmartMeterMonitor.sh              # Start Script
```

Die Bibliotheken für MQTT und Configparser sind vollständig im Repository enthalten und werden direkt im Source eingebunden. Die Bibliothek für Modbus ist nicht Bestandteil des Repositories. Sie muss separat von 'https://github.com/stephane/libmodbus' heruntergeladen und in den Projektbaum entpackt werden. Die Modbus-Bibliothek wird zur Laufzeit dynamisch geladen (libmodbus.so).

Die Applikation wird direkt auf dem Zielsystem erstellt, so dass der oben dargestellte Projektbaum vollständig auf dem Zielsystem vorhanden sein muss.

Die nachfolgenden Schritte beschreiben das Erstellen der Applikation auf einem ***sysWORXX Pi-AM62x*** mit ***Smart Metering HAT***. Diese Plattform nutzt ein robustes Yocto-Linux für den industriellen Einsatz, bei dem das Verzeichnis zur Laufzeit */usr* nicht beschreibbar ist. Daher muss die Bibliothek *libmodbus* so generiert werden, dass sowohl *libmodbus.h* als auch *libmodbus.so* innerhalb des *home* Verzeichnisses installiert werden. Dazu wird das *configue* Skript mit einem entsprechenden Parameter *--prefix=* aufgerufen.

Im Nachfolgenden wird davon ausgegangen, dass auf dem *sysWORXX Pi-AM62x* ein Nutzer *'smm'* mit dem zugehörigen Homeverzeichnis *'/home/smm'* existiert. Die Applikation *'SmartMeterMonitor'* wird in folgendem Pfad erstellt:

```
/home/smm/SmartMeterMonitor
```

Zunächst sind die Sourcen der im Repository nicht enthaltene Modbus-Bibliothek in das Unterverzeichnis *'libmodbus'* zu kopieren. Dies kann entweder direkt mit dem Befehl *'git clone https://github.com/stephane/libmodbus.git'* erfolgen, oder über den Download als ZIP-File von 'https://github.com/stephane/libmodbus'. In diesem Fall wird das Archiv 'libmodbus-master.zip' in das Stammverzeichnis '`/home/smm/SmartMeterMonitor`' abgelegt und wie folgt entpackt:

```
cd /home/smm/SmartMeterMonitor
unzip ./libmodbus-master.zip
mv libmodbus-master libmodbus
```

Nun kann die Bibliothek *'libmodbus'* auf dem Target erstellt werden:

```
cd /home/smm/SmartMeterMonitor/libmodbus
./autogen.sh
./configure --prefix=/home/smm/bin
make
make install
```

Die erstellte Bibliothek *'libmodbus'* sowie das zugehörige Headerfile befindet sich jetzt in folgenden Verzeichnissen:

```
/home/smm/bin/include/modbus      -> modbus.h
/home/smm/bin/lib                 -> libmodbus.so -> libmodbus.so.5.1.0
```

Die typische Installation in '/usr/local/lib' über den Befehl *'sudo make install'* schlägt auf dem *sysWORXX Pi* durch dessen R/O Filesystem fehl.

Falls die Bibliothek *'libmodbus'* in ein anderes Verzeichnis erstellt wurde, sind im *Makefile* der Applikation die Pfade auf die Bibliothek *'libmodbus'* sowie das zugehörige Headerfile anzupassen:

MODBUS_H = /home/smm/bin/include/modbus
MODBUS_LIB = -L/home/smm/bin/lib -lmodbus

Für die GasMeter-Unterstützung ist außerdem im *Makefile* das Define *_GASMETER_* zu setzen.

Nachdem der oben dargestellte Projektbaum inklusive Bibliothek *'libmodbus'* auf dem Target vollständig vorhanden ist, wird die Applikation *'SmartMeterMonitor'* durch einen simplen *make* Aufruf erstellt:

```
cd /home/smm/SmartMeterMonitor/SmartMeterMonitor
make
```

Bei einem abweichenden Installationspfad für die Bibliothek *'libmodbus'* muss im Skript `run_SmartMeterMonitor.sh` gegebenenfalls die Zeile

```
export LD_LIBRARY_PATH=/home/smm/bin/lib
```

angepasst werden. Anschließend kann die Applikation mit dem Skript gestartet werden.

Hinzufügen weiterer SmartMeter-Typen

Um der Applikation um weitere SmartMeter-Typen hinzuzufügen, sind folgende Schritte notwendig:

1. neue Klasse als Implementierung der abstrakten Basisklasse `SmartMeter` für den SmartMeter-Typ ableiten (siehe *SmartMeter.h*, als Vorlage an Klassen `SDM230` und `SDM630` orientieren)
2. neu erstellte Klasse mit dediziertem if-Zweig in die Funktion `BuildSmDeviceList()` einbinden
3. Sourcecode-Datei der neu erstellten Klasse in `Makefile` einbinden

Hardwareanbindung für alternativen Gas-Sensor implementieren

Um die Hardwareanbindung eines alternativen Sensors für den Gaszähler zu implementieren, sind folgende Schritte notwendig:

1. neue Implementierung für die in *GMSensIf.h* definierten Funktionen in separater *.cpp Datei erstellen (an *GMSensIf_sysPi.cpp* orientieren)
2. Im `Makefile` Verweise auf *GMSensIf_sysPi.** durch Verweise auf die neu erstellte Sourcecode-Datei ersetzen

Inbetriebnahmeunterstützung für Gaszähler

Der Sensor für den Gaszähler besteht nur aus einem einfachen Read-Kontakt, der von einem Magneten im mechanischen Zählwerk des Gaszählers bei jeder Umrundung getriggert wird. Für eine fehlerfreie Zählfunktion muss der Sensor exakt zum Zählwerk ausgerichtet sein. Um die korrekte Funktion überprüfen zu können, ist in der Hardwareanbindung *GMSensIf_sysPi.cpp* ein "Impuls-Analysator" implementiert. Dieser wird über die Zeile

```
#define _SENSOR_IMPULSE_ANALYZER_
```

am Anfang der Sourcecode-Datei aktiviert bzw. deaktiviert. Bei aktivem "Impuls-Analysator" wird jeder registrierte Zählimpuls in einer csv-Datei mit Zeitstempel und aktuellem Zählerwert protokolliert:

```
SensorCounterValue;TimeStamp
3956124;26173991
3956125;26206245
3956126;26230844
3956127;26251704
```

Pfad und Name der csv-Datei werden direkt innerhalb von *GMSensIf_sysPi.cpp* definiert:

```
static const char*  ANALYZER_FILE_PATH_NAME = "/tmp/SensorImpAnalyzer.log";
```

Durch Vergleich des von der Applikation ermittelten Zählerwertes mit dem jeweiligen Zählerstand des mechanischen Zählwerkes des Gaszählers kann die korrekte Zählfunktion des Sensors überprüft werden.

Verwendete Drittanbieter Komponenten

1. **Modbus/RTU**

   Für die RS485-basierte Modbus/RTU Kommunikation mit den SmartMetern wird die Bibliothek *"libmodbus"* verwendet:
   https://github.com/stephane/libmodbus

2. **MQTT Client**

   Für die MQTT Client Implementierung wird die *"Paho MQTT Embedded/C"* Bibliothek verwendet:
   https://github.com/eclipse/paho.mqtt.embedded-c

3. **INI-File Parser**

   Zum Lesen der Konfigurationseinstellungen aus der INI-Datei (typisch `config.ini`) wird die Bibliothek *"configparser"*:
   https://github.com/dzilles/configparser

   Die in diesem Repository enthaltenen Sourcen implementieren einen für diese Applikation erweiterten Funktionsumfang der Bibliothek. Die Modifikationen sind durch entsprechende Kommentare gekennzeichnet.